

- Portfolio Write-up — AI Learning Planner
 - Problem
 - Approach
 - Cara Kerja Teknis
 - Keputusan Arsitektur & Trade-off
 - Stack Teknis
 - Fitur Utama yang Diimplementasi
 - Impact
 - Dampak Terukur (Target Cycle 1)
 - Dampak Kualitatif
 - Kontribusi Teknis Pribadi

Portfolio Write-up — AI Learning Planner

Format: Problem → Approach → Impact

Audience: Technical recruiter / hiring manager

Project: Capstone Dicoding Bootcamp · Mei 2026

Problem

Peserta bootcamp Dicoding umumnya memiliki tujuan belajar yang jelas — misalnya "kuasai React dalam 4 minggu" — tetapi **gagal menerjemahkannya menjadi jadwal mingguan yang realistis dan konsisten**. Ada tiga penyebab utama yang teridentifikasi:

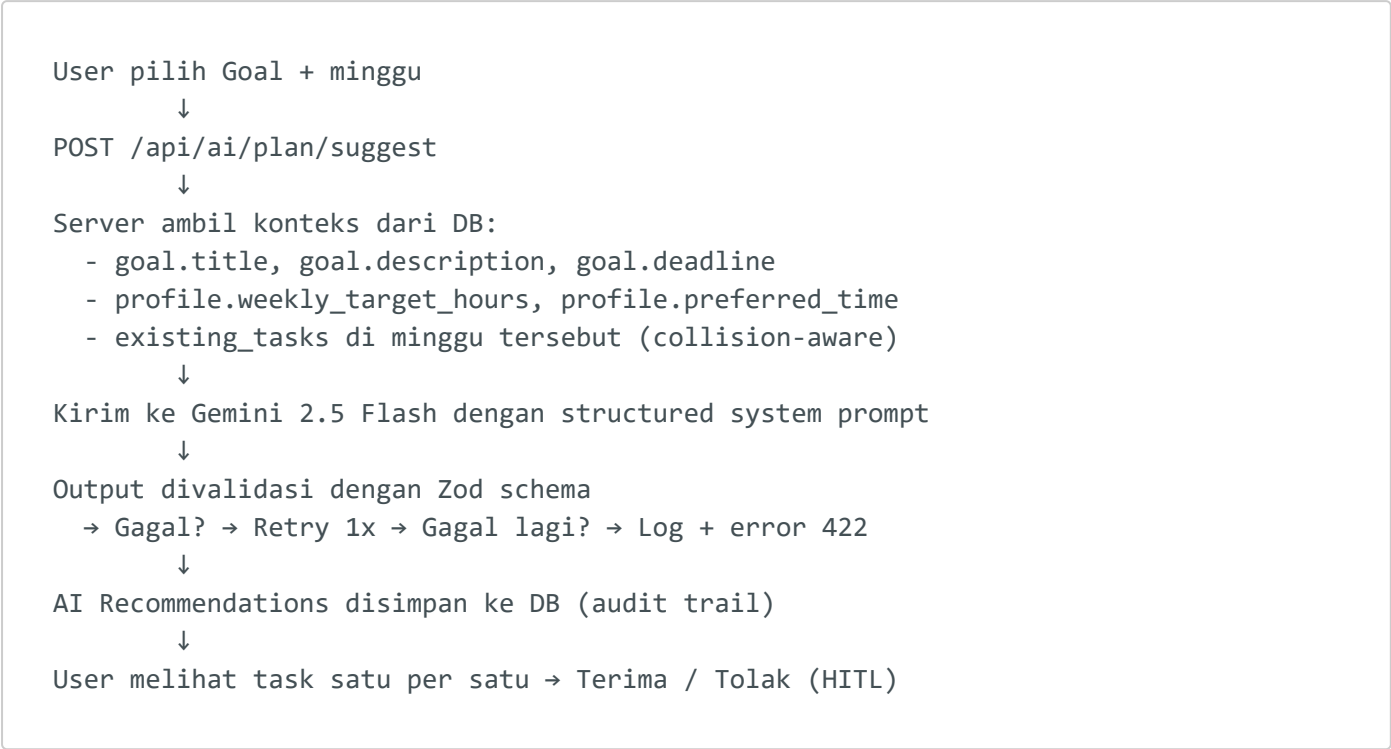
1. **Goal terlalu besar, tidak terpecah menjadi task konkret.** Tanpa panduan granular, peserta tidak tahu harus mulai dari mana, kapan, dan berapa lama.
2. **Jadwal tidak memperhitungkan kapasitas nyata.** Rencana dibuat tanpa mempertimbangkan sesi belajar yang sudah ada di minggu yang sama, sehingga cepat overload dan ditinggalkan.
3. **Tidak ada mekanisme adaptif.** Ketika satu sesi terlewat, tidak ada sistem yang membantu penyesuaian. Peserta akhirnya menyerah alih-alih menjadwalkan ulang.

Akibatnya, **tingkat penyelesaian rencana belajar mingguan tetap rendah** — bukan karena kurang motivasi, melainkan karena sistem perencanaannya tidak cukup membantu pengguna untuk memulai dan beradaptasi.

Approach

Solusi yang dibangun adalah **AI-powered weekly learning planner**: aplikasi web full-stack yang menghasilkan rencana belajar mingguan yang dipersonalisasi berbasis konteks nyata dari database, bukan template statis.

Cara Kerja Teknis



Keputusan Arsitektur & Trade-off

Keputusan	Alasan	Trade-off
Schema-first output validation (Zod)	LLM tidak selalu menghasilkan JSON yang valid; validasi ketat di server mencegah data corrupt masuk ke DB	Sedikit overhead pada setiap response AI, tapi eliminasi bug runtime lebih berharga

Keputusan	Alasan	Trade-off
Retry-once pattern	Satu retry cukup mengatasi fluktuasi LLM tanpa menambah latency berlebihan	Kegagalan permanen tetap terjadi; dicatat di log Pino untuk monitoring
Human-in-the-loop (HITL)	AI hanya memberikan saran, user punya kendali penuh untuk menerima/menolak	Menambah friction dibanding auto-apply, tapi mencegah jadwal diisi tanpa persetujuan user
Privacy-first context	Context yang dikirim ke Gemini tidak mengandung email, nama, atau user ID — hanya data belajar	Default by design, bukan opsi tambahan
Audit trail by default	Setiap output LLM disimpan ke tabel <code>ai_recommendations</code> terlepas dari keputusan user	Storage overhead kecil, tapi menghasilkan dataset untuk analisis pola dan iterasi prompt
Mock LLM mode	<code>LLM_PROVIDER=mock</code> mengembalikan data statis tanpa memanggil Gemini API	Mempercepat iterasi development tanpa biaya API; dinonaktifkan otomatis di production
Conflict-aware scheduling	Saat reschedule, server mendeteksi slot yang sudah terisi dan mengirim <code>occupied_slots</code> ke LLM pada retry berikutnya	AI tetap bisa clash pada attempt pertama; retry dengan konteks tambahan memitigasi ini

Stack Teknis

Layer	Teknologi
Frontend	React (Vite), React Router, CSS Modules
Backend	Node.js 20, Express
LLM Integration	Google Gemini 2.5 Flash via <code>@google/generative-ai</code>
Output Validation	Zod (schema-first, server-side)
Database	PostgreSQL 16 (via node-postgres)

Layer	Teknologi
Auth	JWT + bcrypt
Observability	Pino (structured logging), prom-client (Prometheus metrics)
Infrastructure	Docker Compose (PostgreSQL + Redis + server + client)

Fitur Utama yang Diimplementasi

- **AI Suggestion Flow:** `POST /api/ai/plan/suggest` — generate rencana mingguan berdasarkan konteks goal, profil, dan existing tasks
- **AI Reschedule:** `POST /api/ai/plan/reschedule` — jadwalkan ulang task overdue ke minggu berjalan dengan conflict detection
- **Weekly Calendar View:** visualisasi task per hari dengan slot (pagi/siang/malam) untuk navigasi antar minggu
- **Progress Dashboard:** ringkasan mingguan real-time (task selesai, jam belajar, goal aktif + persentase)
- **Manual Task Reschedule:** form inline di GoalDetail untuk edit tanggal dan slot task tanpa AI
- **State machine transisi status:** validasi transisi ketat (`todo` → `in_progress` → `done/skipped`) di server

Impact

Dampak Terukur (Target Cycle 1)

Metrik	Target	Cara Ukur
Acceptance rate saran AI	≥ 50% saran diterima	Rasio <code>status = 'accepted'</code> vs total di <code>ai_recommendations</code>
Valid output rate	≥ 90% response LLM lolos validasi Zod tanpa retry	Log <code>ai_suggest_failed</code> vs total request
Latency endpoint AI	Median response < 5 detik	Pino request log + prom-client histogram

Dampak Kualitatif

- **Eliminasi blank-page problem:** peserta tidak perlu merancang jadwal dari nol — AI menyediakan titik awal konkret yang bisa diterima atau dimodifikasi.
- **Kapasitas yang realistis:** rencana yang dihasilkan memperhitungkan existing tasks di minggu yang sama, bukan asumsi kosong.
- **Adaptasi saat overdue:** fitur reschedule memungkinkan peserta recovery dari task yang terlewat tanpa harus memulai dari awal.
- **Data untuk iterasi:** pola penerimaan/penolakan yang terekam di `ai_recommendations` menjadi dataset untuk memperbaiki prompt di iterasi berikutnya.

Kontribusi Teknis Pribadi

- Merancang dan mengimplementasi **AI suggestion engine** end-to-end: dari context assembly, LLM call, Zod validation, retry logic, hingga audit trail ke database.
- Membangun **conflict-aware reschedule flow** dengan iterative context enrichment (occupied slots dikirim ulang ke LLM pada retry).
- Mengimplementasi **progress snapshot system** dengan recalculation otomatis setiap kali task dibuat, diupdate, atau direschedule.
- Menyusun **Docker Compose setup** untuk environment development yang reproduibel (PostgreSQL + Redis + server + client dalam satu command).
- Menerapkan **structured logging** (Pino) dan **metrics endpoint** (prom-client) sebagai fondasi observability production.